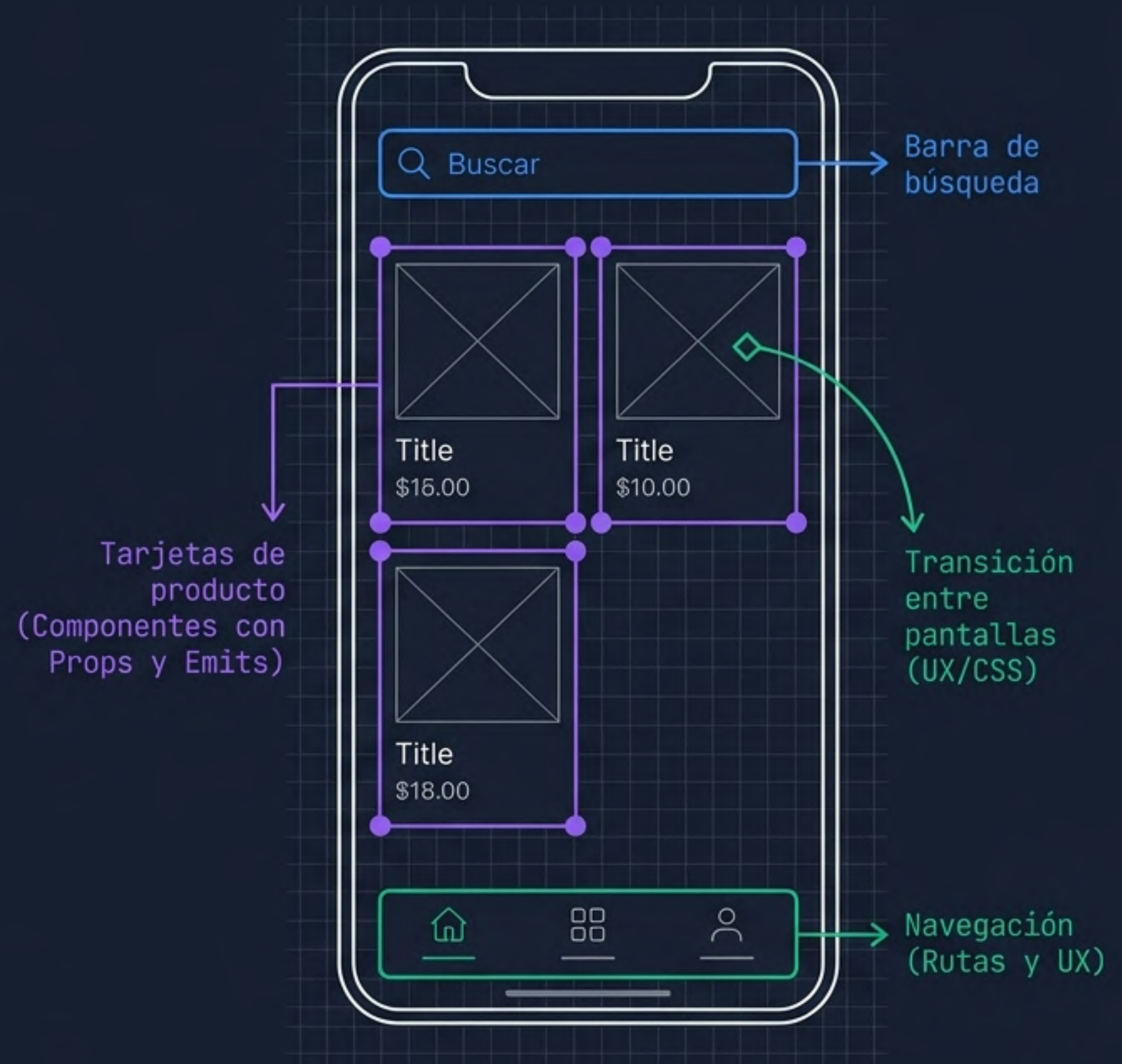
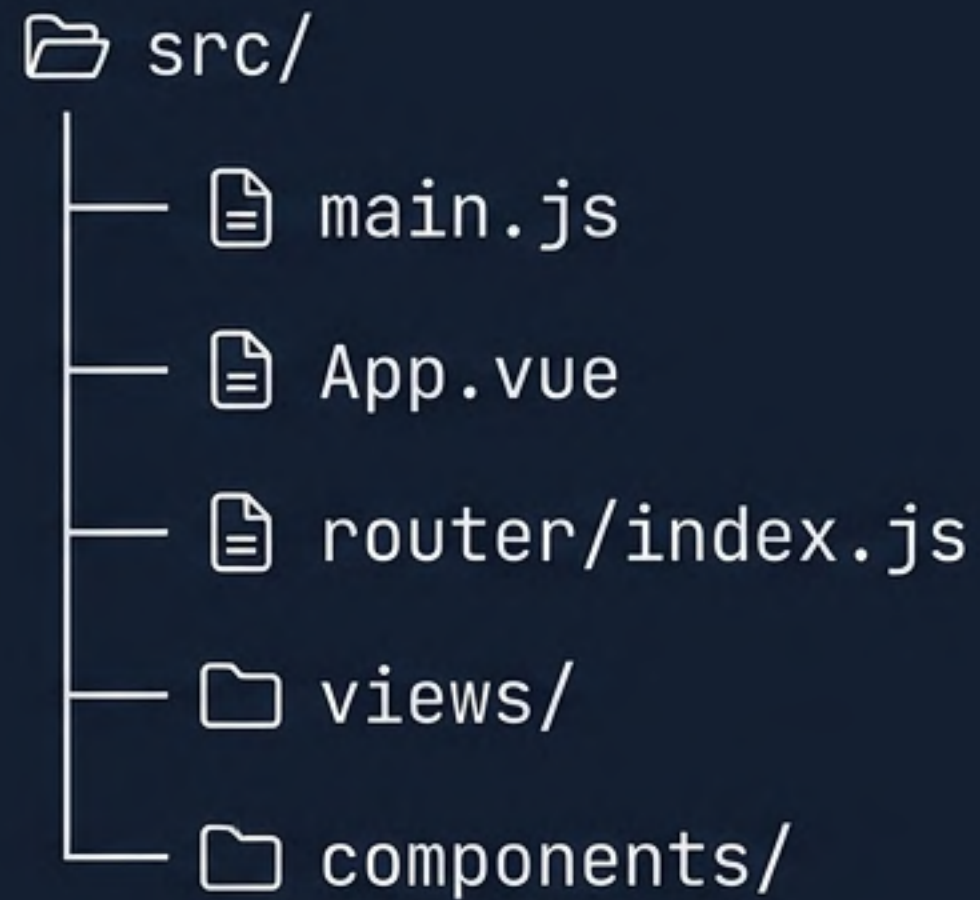


Vue 3 Composition API

Estado, componentes,
rutas y transiciones.

Contexto: Construcción de una
mini app de productos.





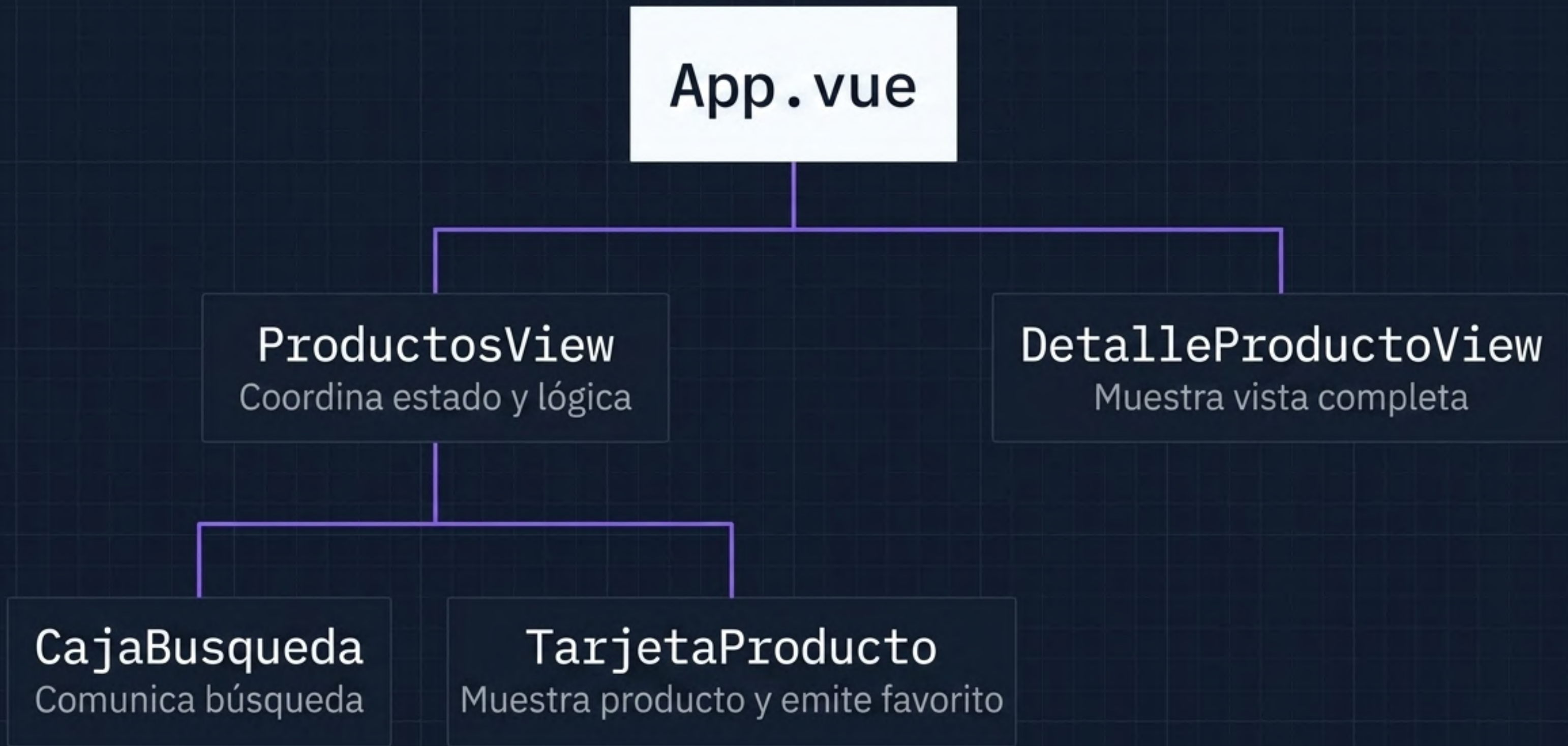
```
import { createApp } from 'vue'
import App from './App.vue'
import router from './router'

createApp(App).use(router).mount('#app')
```

Instancia

Router

Montaje



Idea Clave: La aplicación se organiza como un árbol dinámico de componentes.

Estado Reactivo: ref()

Contenedor reactivo profundo para datos que cambian y actualizan la vista automáticamente.

ref() crea una referencia reactiva. Cuando cambia .value, Vue detecta el cambio y actualiza el template.

En <script> se usa .value; en el template se usa directo.

Sirve para valores que cambian durante la interacción del usuario.

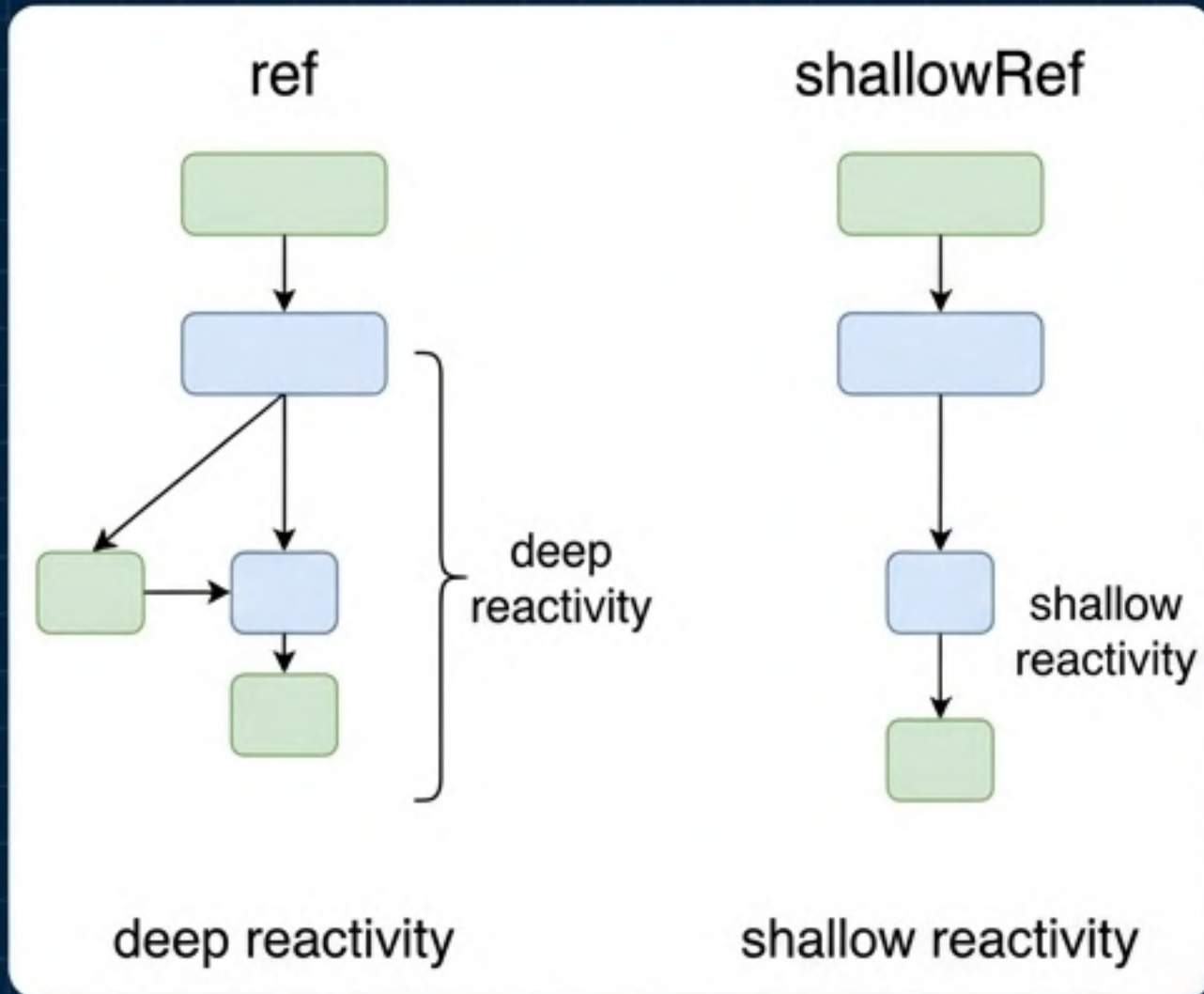
```
<script setup>
import { ref } from 'vue'

const contador = ref(0)

function incrementar() {
  contador.value++
}
</script>

<template>
  <p>{{ contador }}</p>
  <button @click="incrementar">Sumar</button>
</template>
```


Rendimiento: shallowRef()



Referencia reactiva superficial. Reacciona al reemplazar el valor completo, optimizando listas grandes.

Contrasta con `ref()` para entender su uso.

- **`ref()`**: hace reactivo el contenido interno.
- **`shallowRef()`**: solo observa el primer nivel.
- Si se modifica una propiedad interna, Vue no necesariamente re-renderiza.
- Si se reemplaza el array completo, sí reacciona.
- En este ejemplo sirve porque la lista se actualiza reemplazándola con **`filter()`**, **`map()`** o **`spread`**.

Ejemplo comparativo:

```
const productos = shallowRef([...])

// Correcto con shallowRef:
productos.value = productos.value.map(p =>
  p.id === 1 ? { ...p, precio: 9999 } : p
)
```

`shallowRef` funciona mejor cuando reemplazamos el array completo, no cuando mutamos propiedades internas.

Estado Derivado: computed()

Estado derivado en caché. Se recalcula automáticamente si sus dependencias reactivas cambian.

- computed() no reemplaza a una variable normal.
- Se usa cuando un dato depende de otro.
- Evita guardar manualmente productosFiltrados como otro estado.
- Si cambia búsqueda o productos, Vue recalcula automáticamente.
- Si no cambian sus dependencias, Vue reutiliza el resultado en caché.

Idea Clave: Si un valor se puede calcular desde otros datos reactivos, probablemente debería ser computed.

```
import { computed } from 'vue'

const productosFiltrados = computed(() => {
  if (!busqueda.value.trim()) return
  productos.value

  return productos.value.filter(p =>
    p.nombre.toLowerCase().includes(busqueda.value.toLowerCase()))
})
```


Componentes Reutilizables

Piezas independientes de interfaz.

Idea Clave: La vista coordina el estado general; los componentes muestran la UI y comunican acciones.



 `CajaBusqueda.vue`

Maneja el input de texto.

 `TarjetaProducto.vue`

Renderiza imagen y precio.

 `DetalleProductoView.vue`

Despliega información extendida.

Props (Padre a Hijo)

Datos de solo lectura enviados desde un componente padre hacia uno hijo.
Flujo unidireccional.



ProductosView.vue

```
<TarjetaProducto :producto='producto' />
```

TarjetaProducto.vue

```
const props = defineProps({
  producto: Object
})

// Uso en template:
<h3>{{ producto.nombre }}</h3>
<p>${{ producto.precio }}</p>
```

Atención:

El hijo no debe modificar directamente la prop.
Si necesita cambiar algo, debe emitir un evento al padre.

Emits (Hijo a Padre)

- El hijo no cambia directamente totalFavoritos.
- El hijo solo avisa: "ocurrió esta acción".
- El padre decide qué hacer con ese evento.
- Esto mantiene el estado centralizado en la vista.



TarjetaProducto.vue

```
const emit = defineEmits(['favorito'])

function agregarFavorito() {
  emit('favorito')
}
```

ProductosView.vue

```
<TarjetaProducto @favorito='sumarFavorito' />
```


Vue Router

Librería oficial para navegación SPA.

Permite navegar entre vistas sin recargar la página, actualizando dinámicamente el componente mostrado según la URL.

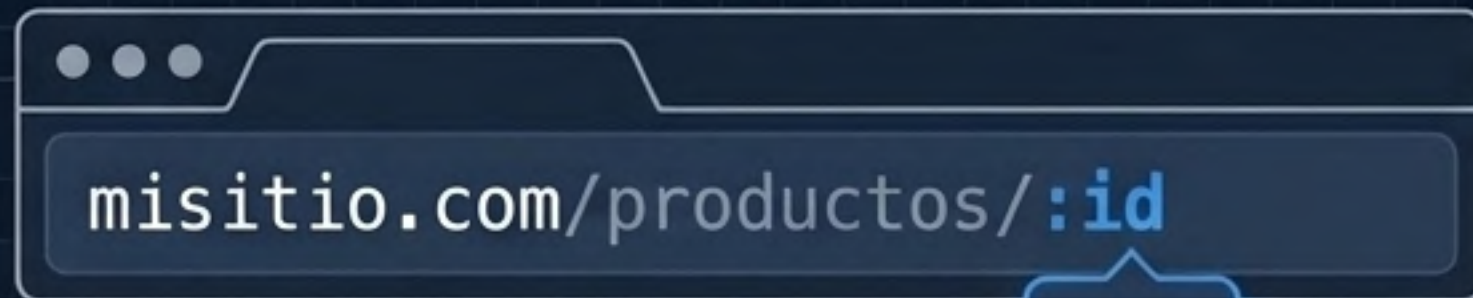
- En una SPA no se recarga toda la página al navegar.
- Vue Router cambia el componente mostrado según la URL.
- `<RouterLink>` crea enlaces internos.
- `<RouterView>` muestra la vista correspondiente.
- `createWebHistory()` permite URLs limpias como `/productos/1`.

```
<nav>
  <RouterLink to="/">Inicio</RouterLink>
  <RouterLink to="/productos">Productos</RouterLink>
</nav>

<RouterView />
```


Rutas Dinámicas

Una ruta dinámica permite reutilizar una misma vista para muchos elementos. `:id` funciona como una variable dentro de la URL. `/productos/1`, `/productos/2` y `/productos/3` usan el mismo componente. Lo que cambia es el parámetro recibido.



Idea clave: "No necesitamos una vista distinta para cada producto; usamos una sola vista dinámica."

1

2

3

```
// En router/index.js
{
  path: '/productos/:id',
  component: DetalleProductoView,
  props: true
}
```

`/productos/1` y `/productos/2`
usan la misma vista exacta.

Buscar Producto por ID

Captura el parámetro de ruta (como String) y usa estado derivado para aislar el producto correcto.



```
const props = defineProps({
  id: String
})

const producto = computed(() =>
  productos.value.find(p => p.id
    === Number(props.id))
)

<template>
  <p v-if="!producto">Producto no
    encontrado</p>
  <section v-else>
    <h2>{{ producto.nombre }}</h2>
    <p>{{ producto.precio }}</p>
  </section>
</template>
```


El Sistema de Clases de <Transition>

- Vue no anima por sí solo: Vue agrega clases en momentos específicos.
- El CSS define cómo se ve la animación.
- name='fade' genera clases como: fade-enter-active, fade-enter-from, fade-enter-to, fade-leave-active, fade-leave-from, fade-leave-to.
- Para que Transition actúe, Vue debe detectar entrada o salida del elemento.

Idea clave: "Vue controla cuándo se aplican las clases; CSS controla cómo se anima."

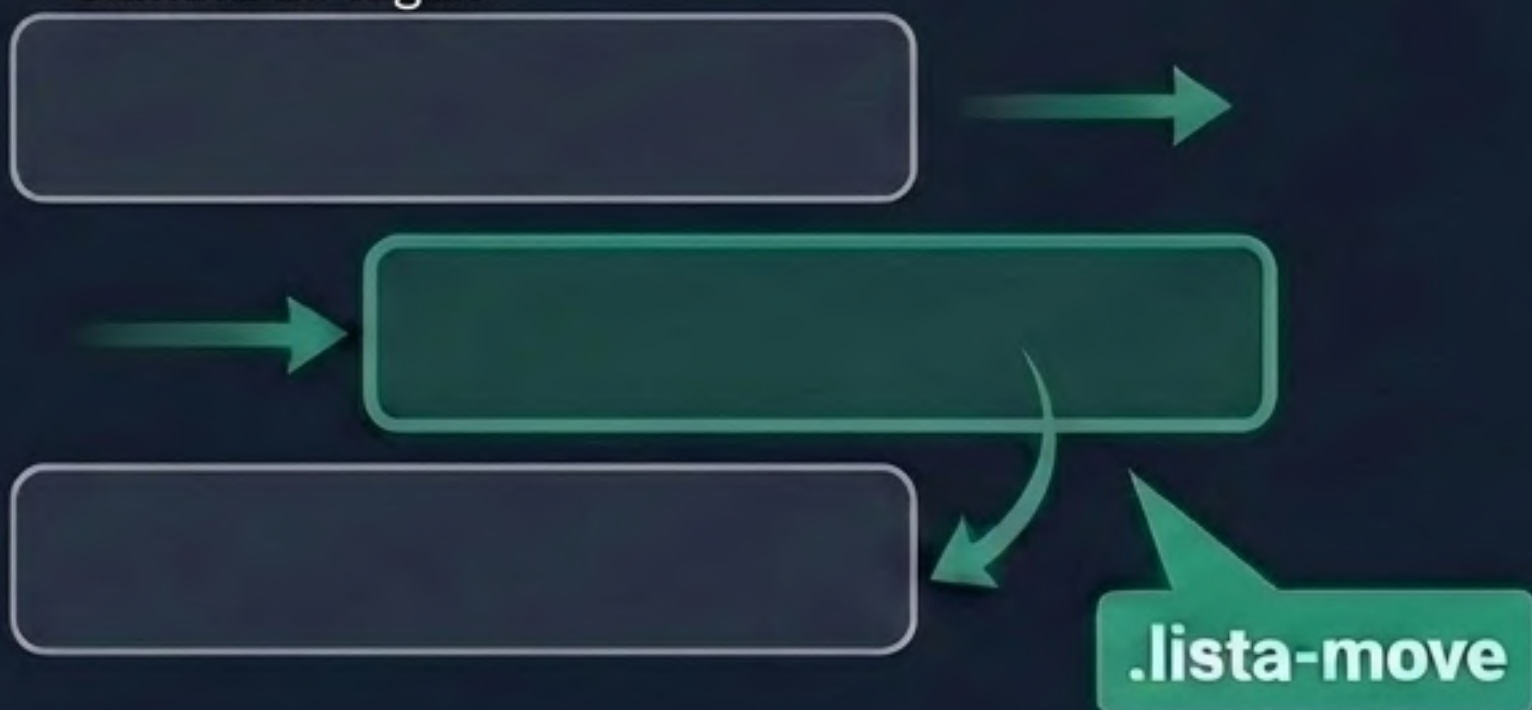
```
<Transition name='fade'>
  <p v-if='mostrar'>Producto agregado</p>
</Transition>
```

```
.fade-enter-active,
.fade-leave-active {
  transition: opacity .3s ease;
}
.fade-enter-from,
.fade-leave-to {
  opacity: 0;
}
```


Coreografías:

<TransitionGroup>

- **Transition** anima un solo elemento.
- **TransitionGroup** anima varios elementos dentro de una lista.
- Necesita **:key** para identificar cada elemento.
- La clase **-move** se usa cuando un elemento se reordena o cambia de lugar.



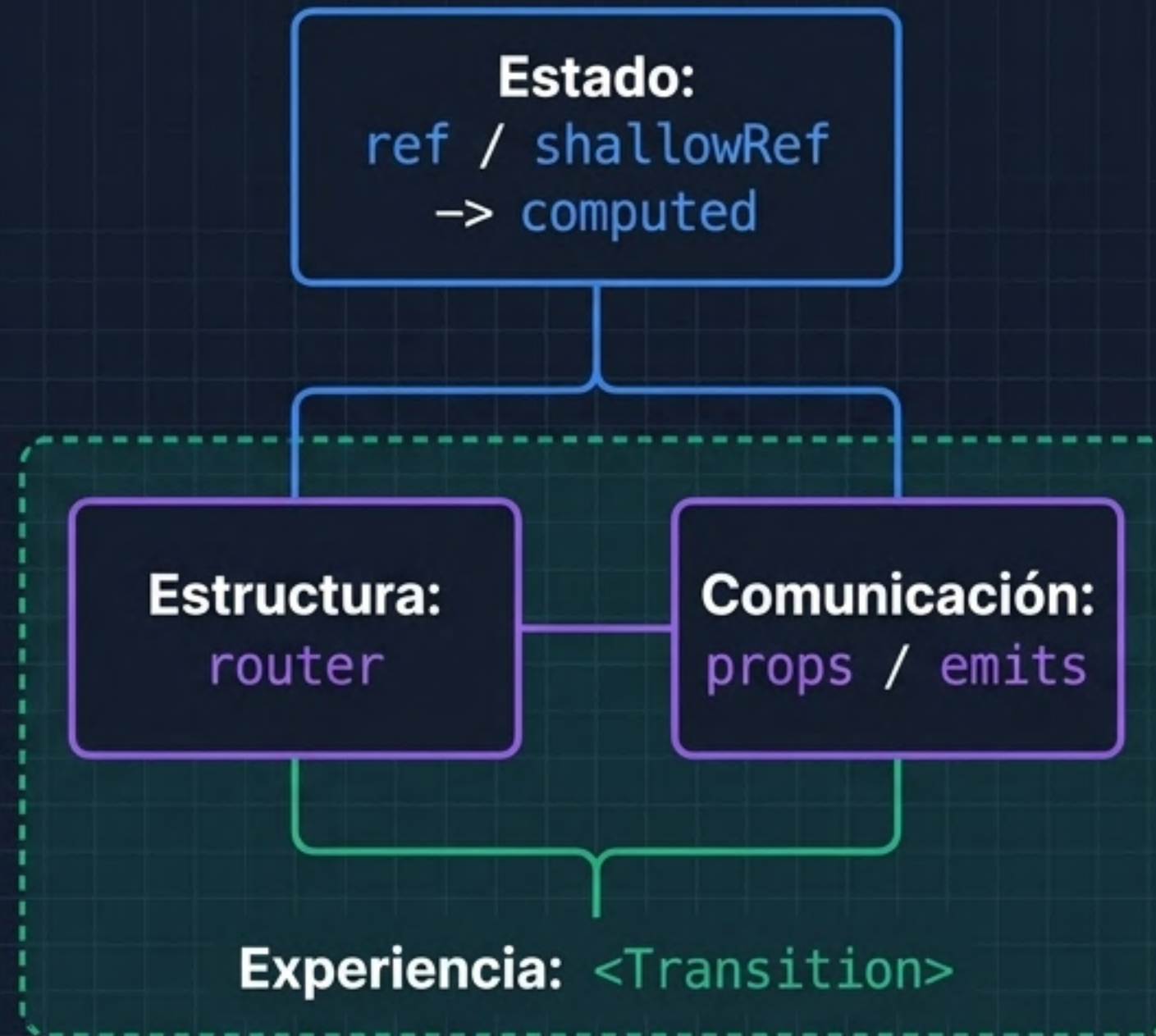
Idea clave: “Cuando se filtra la lista, Vue compara las keys y aplica clases de transición a los productos que entran, salen o se mueven.”

```
<TransitionGroup name='lista' tag='div'>
  <TarjetaProducto
    v-for='producto in productosFiltrados'
    :key='producto.id'
    :producto='producto'
  />
</TransitionGroup>
```

```
.lista-enter-active,
.lista-leave-active,
.lista-move {
  transition: all .3s ease;
}
.lista-enter-from,
.lista-leave-to {
  opacity: 0; transform: translateY(12px);
}
```


Síntesis: La Arquitectura de Vue 3

- **Estado:** `ref` y `shallowRef` almacenan; `computed` deriva.
- **Comunicación:** `props` bajan datos; `emits` suben eventos.
- **Estructura:** `router` gestiona vistas dinámicas.
- **Experiencia:** `Transition` y `TransitionGroup` inyectan fluidez.



Reflexión Final

¿Qué conceptos trabajan juntos cuando el usuario filtra la lista de productos?

(Mutación de `ref` -> Recálculo de `computed` -> Animación vía `<TransitionGroup>`)